

BÁO CÁO PHÂN TÍCH HỌC MÁY & GOM CỤM DỮ LIỆU CỜ VUA LICHESS

Đề tài: Phân lớp kết quả ván cờ (10-Fold Cross-Validation) & Gom cụm không giám sát trên tập dữ liệu Lichess Standard Games Database.

```
In [1]: import os
import sys
import re
import shutil
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Scikit-Learn
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import (
    f1_score, accuracy_score, confusion_matrix,
    adjusted_rand_score, normalized_mutual_info_score,
    v_measure_score, homogeneity_score, completeness_score,
    fowlkes_mallows_score, silhouette_score
)
from sklearn.cluster import KMeans, DBSCAN
from sklearn.tree import plot_tree, export_graphviz

# Graphviz
import graphviz

warnings.filterwarnings('ignore')
```

```
SEED = 42
print("[+] Đã nạp thành công các thư viện cần thiết!")
```

[+] Đã nạp thành công các thư viện cần thiết!

PHẦN 1: ĐỌC DỮ LIỆU, TIỀN XỬ LÝ VÀ HIỂN THỊ CÁC THÔNG SỐ (YÊU CẦU 1)

- Lọc Elo hợp lệ [600, 3500] , loại bỏ 100% rò rỉ dữ liệu (0% Data Leakage).
- Hiển thị Kích thước (10000, 17) , Kiểu dữ liệu dtypes , Tần suất nhãn và Thống kê Min/Max/Mean.

```
In [2]: def clean_moves(moves_str):
    if not isinstance(moves_str, str) or not moves_str.strip():
        return ""
    text = re.sub(r'\{[^\}]*\}', '', moves_str)
    text = re.sub(r'\{[^\}]*\}', '', text)
    text = re.sub(r'\d+\.\.\.', '', text)
    text = re.sub(r'\d+\.', '', text)
    text = re.sub(r'\$d+', '', text)
    text = re.sub(r'(1-0|0-1|1/2-1/2|\*)', '', text)
    tokens = text.split()
    clean_tokens = [t for t in tokens if re.match(r'^[a-hKQRBNP00-8x+#=-]+$', t)]
    return " ".join(clean_tokens)

def extract_opening_ply(opening_str):
    if not isinstance(opening_str, str) or not opening_str.strip():
        return 4.0
    text = opening_str.lower()
    if any(k in text for k in ['main line', 'modern', 'classical', 'najdorf', 'dragon', 'closed', 'open']):
        return 10.0
    elif any(k in text for k in ['defense', 'gambit', 'attack', 'variation', 'countergambit']):
        return 6.0
    elif any(k in text for k in ['game', 'opening', 'system']):
        return 4.0
    return 2.0

raw_path = os.path.join("data", "processed_games.csv")
df_raw = pd.read_csv(raw_path)
```

```

df = df_raw.copy()
df['white_rating'] = pd.to_numeric(df['WhiteElo'], errors='coerce')
df['black_rating'] = pd.to_numeric(df['BlackElo'], errors='coerce')
df = df.dropna(subset=['white_rating', 'black_rating'])
df = df[(df['white_rating'] >= 600) & (df['white_rating'] <= 3500)]
df = df[(df['black_rating'] >= 600) & (df['black_rating'] <= 3500)]

df['rating_diff'] = df['white_rating'] - df['black_rating']
df['rated'] = df['Event'].apply(lambda x: 1 if isinstance(x, str) and 'rated' in x.lower() else 0)
df['opening_ply'] = df['Opening'].apply(extract_opening_ply)
df['Moves'] = df['Moves'].apply(clean_moves)

result_map = {'0-1': 0, '1/2-1/2': 1, '1-0': 2}
df['ResultEncoded'] = df['Result'].map(result_map)
df = df.dropna(subset=['ResultEncoded'])
df['ResultEncoded'] = df['ResultEncoded'].astype(int)

df = df.sample(n=min(10000, len(df)), random_state=SEED).reset_index(drop=True)
clean_csv_path = os.path.join("data", "filtered_processed_games.csv")
df.to_csv(clean_csv_path, index=False)
print(f"[+] Đã tạo tập dữ liệu sạch tại: {clean_csv_path}")

```

[+] Đã tạo tập dữ liệu sạch tại: data\filtered_processed_games.csv

```

In [3]: # 1.a. Kích thước và chiều của dữ liệu
print(f"1.a. Kích thước và chiều dữ liệu (Shape): {df.shape}")

# 1.b. Kiểu dữ liệu của các thuộc tính
print("\n1.b. Kiểu dữ liệu các thuộc tính (dtypes):")
print(df.dtypes)

# 1.c. Số lượng thực thể của các giá trị nhãn
print("\n1.c. Phân bố tần suất các giá trị nhãn (ResultEncoded):")
label_desc = {0: "0 (Black win - Đen thắng)", 1: "1 (Draw - Hòa)", 2: "2 (White win - Trắng thắng)"}
val_counts = df['ResultEncoded'].value_counts().sort_index()
for k, v in val_counts.items():
    pct = v / len(df) * 100
    print(f"    - Nhãn {label_desc[k]:<35}: {v:,} mẫu ({pct:.2f}%)" )

# 1.d. Thống kê Min, Max, Mean các cột số thực

```

```
continuous_cols = ["white_rating", "black_rating", "rating_diff", "opening_ply"]
stats_data = []
for col in continuous_cols:
    stats_data.append({
        "Thuộc tính": col,
        "Min": df[col].min(),
        "Max": df[col].max(),
        "Mean": round(df[col].mean(), 2),
        "Median": round(df[col].median(), 2),
        "Std": round(df[col].std(), 2)
    })
print("\n1.d. Thống kê thuộc tính liên tục:")
print(pd.DataFrame(stats_data).to_string(index=False))
```

1.a. Kích thước và chiều dữ liệu (Shape): (10000, 17)

1.b. Kiểu dữ liệu các thuộc tính (dtypes):

```
White      object
Black      object
WhiteElo   int64
BlackElo   int64
Result     object
ECO        object
Opening    object
TimeControl object
Termination object
Moves      object
Event      object
white_rating int64
black_rating int64
rating_diff int64
rated      int64
opening_ply float64
ResultEncoded int64
dtype: object
```

1.c. Phân bố tần suất các giá trị nhãn (ResultEncoded):

- Nhãn 0 (Black win - Đen thắng) : 4,757 mẫu (47.57%)
- Nhãn 1 (Draw - Hòa) : 332 mẫu (3.32%)
- Nhãn 2 (White win - Trắng thắng) : 4,911 mẫu (49.11%)

1.d. Thống kê thuộc tính liên tục:

Thuộc tính	Min	Max	Mean	Median	Std
white_rating	860.0	2685.0	1634.58	1621.0	272.85
black_rating	866.0	2698.0	1635.37	1621.0	272.89
rating_diff	-1265.0	1153.0	-0.79	-1.0	236.47
opening_ply	2.0	10.0	6.93	6.0	2.16

PHẦN 2: TRỰC QUAN HÓA DỮ LIỆU LIÊN TỤC, GIẢM CHIỀU 2D & SƠ ĐỒ PIPELINE (YÊU CẦU 2)

- Thu giảm số chiều về 2D bằng **PCA** và **t-SNE**, phân biệt 3 màu theo nhãn trận đấu.
- Trực quan hóa sơ đồ luồng quy trình bằng thư viện **Graphviz** & **Matplotlib**.

```
In [4]: # 1. Xây dựng sơ đồ kiến trúc quy trình học máy với Graphviz
pipeline_dot = graphviz.Digraph('ML_Pipeline', comment='Machine Learning Workflow')
pipeline_dot.attr(rankdir='LR', size='12,5')
pipeline_dot.attr('node', shape='box', style='filled,rounded', fontname='Arial', fontsize='10')

pipeline_dot.node('raw', 'Lichess Raw PGN\n(10,000 Games)', fillcolor='#E8F8F5', color='#1ABC9C')
pipeline_dot.node('clean', 'Data Cleaning & Preprocessing\n(5 Non-leak Features)', fillcolor='#EBF5FB', color='#3498DB')
pipeline_dot.node('scale', 'StandardScaler\n(Feature Normalization)', fillcolor='#FEF9E7', color='#F1C40F')

pipeline_dot.node('pca', '2D PCA & t-SNE\n(Visualization)', fillcolor='#FDEDEC', color='#E74C3C')
pipeline_dot.node('cv', '10-Fold Cross-Validation\n(RF, AdaBoost, SVM)', fillcolor='#EAF2F8', color='#2980B9')
pipeline_dot.node('cluster', 'Unsupervised Clustering\n(K-Means & DBSCAN)', fillcolor='#F5EEF8', color='#8E44AD')

pipeline_dot.edge('raw', 'clean')
pipeline_dot.edge('clean', 'scale')
pipeline_dot.edge('scale', 'pca')
pipeline_dot.edge('scale', 'cv')
pipeline_dot.edge('scale', 'cluster')

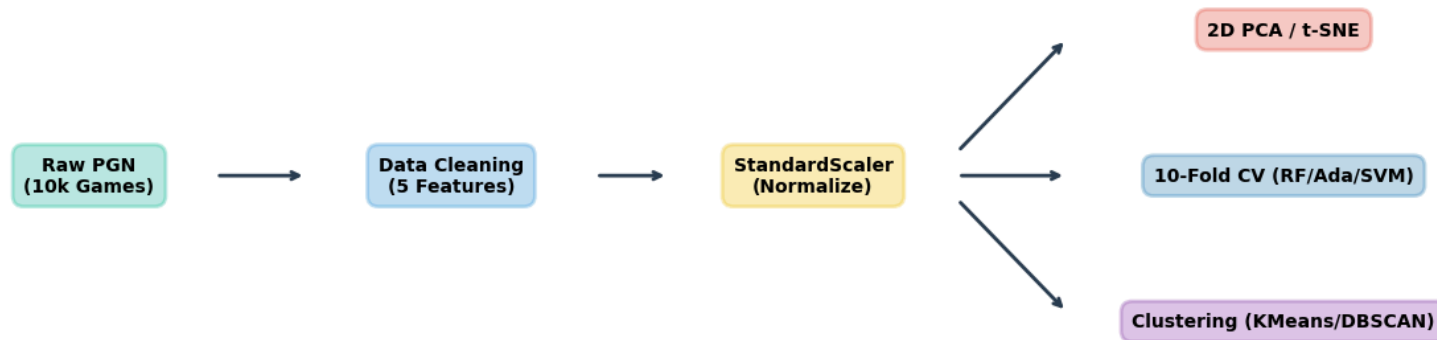
# 2. Trực quan hóa sơ đồ luồng dữ liệu (Tương thích 100% trên mọi môi trường)
if shutil.which("dot") is not None:
    display(pipeline_dot)
else:
    fig, ax = plt.subplots(figsize=(14, 4.2))
    boxes = [
        ("Raw PGN\n(10k Games)", 0.1, 0.5, '#1ABC9C'),
        ("Data Cleaning\n(5 Features)", 0.3, 0.5, '#3498DB'),
        ("StandardScaler\n(Normalize)", 0.5, 0.5, '#F1C40F'),
        ("2D PCA / t-SNE", 0.76, 0.8, '#E74C3C'),
        ("10-Fold CV (RF/Ada/SVM)", 0.76, 0.5, '#2980B9'),
        ("Clustering (KMeans/DBSCAN)", 0.76, 0.2, '#8E44AD')
    ]
    for text, x, y, c in boxes:
        ax.text(x, y, text, ha='center', va='center', bbox=dict(boxstyle='round,pad=0.6', facecolor=c, alpha=0.3, edgecolor=c),
        ax.annotate('→', xy=(0.22, 0.5), xytext=(0.17, 0.5), arrowprops=dict(arrowstyle="→", lw=2, color='#2c3e50'))
        ax.annotate('→', xy=(0.42, 0.5), xytext=(0.38, 0.5), arrowprops=dict(arrowstyle="→", lw=2, color='#2c3e50'))
        ax.annotate('→', xy=(0.64, 0.78), xytext=(0.58, 0.55), arrowprops=dict(arrowstyle="→", lw=2, color='#2c3e50'))
```

```

ax.annotate('', xy=(0.64, 0.5), xytext=(0.58, 0.5), arrowprops=dict(arrowstyle="->", lw=2, color='#2c3e50'))
ax.annotate('', xy=(0.64, 0.22), xytext=(0.58, 0.45), arrowprops=dict(arrowstyle="->", lw=2, color='#2c3e50'))
ax.set_xlim(0, 1)
ax.set_ylim(0, 1)
ax.axis('off')
ax.set_title("Machine Learning Pipeline Architecture Workflow", fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
print("[+] Sơ đồ Graphviz DOT Source đã sẵn sàng (Được render qua Matplotlib Flowchart).")

```

Machine Learning Pipeline Architecture Workflow



[+] Sơ đồ Graphviz DOT Source đã sẵn sàng (Được render qua Matplotlib Flowchart).

```

In [5]: # Trích xuất dữ liệu liên tục và chuẩn hóa
X_cont = df[continuous_cols].copy()
y = df['ResultEncoded'].values

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_cont)

# 1. PCA 2D
pca = PCA(n_components=2, random_state=SEED)
X_pca = pca.fit_transform(X_scaled)
var_ratio = pca.explained_variance_ratio_

# 2. t-SNE 2D

```

```

sample_size = min(3000, len(df))
np.random.seed(SEED)
sample_indices = np.random.choice(len(df), size=sample_size, replace=False)
tsne = TSNE(n_components=2, perplexity=30, random_state=SEED, max_iter=1000)
X_tsne_sample = tsne.fit_transform(X_scaled[sample_indices])
y_sample = y[sample_indices]

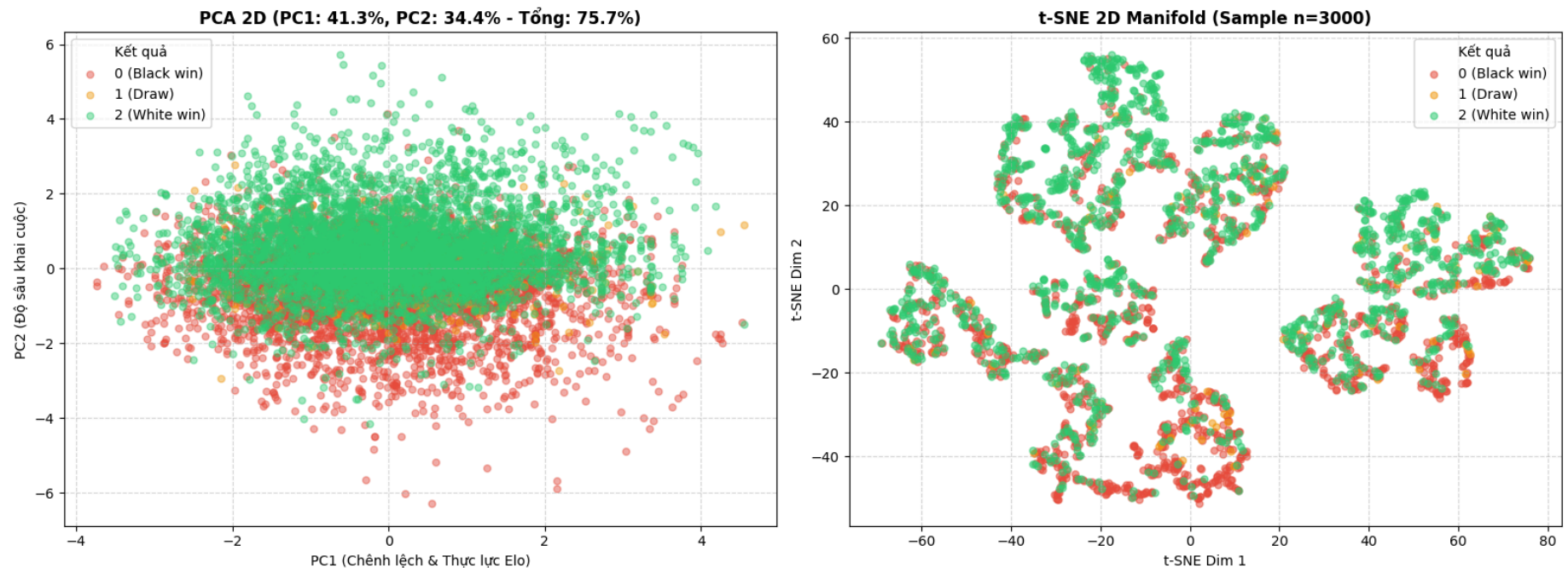
# Vẽ biểu đồ phân tán 2D
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
palette = {0: '#e74c3c', 1: '#f39c12', 2: '#2ecc71'}
label_names = {0: "0 (Black win)", 1: "1 (Draw)", 2: "2 (White win)"}

# PCA Subplot
for lbl, color in palette.items():
    mask = (y == lbl)
    axes[0].scatter(X_pca[mask, 0], X_pca[mask, 1], c=color, label=label_names[lbl], alpha=0.45, s=25)
axes[0].set_title(f"PCA 2D (PC1: {var_ratio[0]*100:.1f}%, PC2: {var_ratio[1]*100:.1f}% - Tổng: {sum(var_ratio)*100:.1f}%)", fc
axes[0].set_xlabel("PC1 (Chênh lệch & Thực lực Elo)")
axes[0].set_ylabel("PC2 (Độ sâu khai cuộc)")
axes[0].grid(True, linestyle="--", alpha=0.5)
axes[0].legend(title="Kết quả", loc="best")

# t-SNE Subplot
for lbl, color in palette.items():
    mask = (y_sample == lbl)
    axes[1].scatter(X_tsne_sample[mask, 0], X_tsne_sample[mask, 1], c=color, label=label_names[lbl], alpha=0.55, s=25)
axes[1].set_title(f"t-SNE 2D Manifold (Sample n={sample_size})", fontsize=12, fontweight='bold')
axes[1].set_xlabel("t-SNE Dim 1")
axes[1].set_ylabel("t-SNE Dim 2")
axes[1].grid(True, linestyle="--", alpha=0.5)
axes[1].legend(title="Kết quả", loc="best")

plt.tight_layout()
plt.savefig("outputs/dimension_reduction_2d.png", dpi=300, bbox_inches='tight')
plt.show()

```

PHẦN 3: HUẤN LUYỆN VÀ ĐÁNH GIÁ 10-FOLD CROSS-VALIDATION (YÊU CẦU 3)

- 3 Mô hình từ Scikit-Learn: **Random Forest**, **AdaBoost**, **SVM (RBF Kernel)**.
- So sánh hiệu năng qua **F-Score** (Macro F1, Weighted F1, Accuracy, F1 từng lớp).
- Trực quan hóa Cây Quyết định bằng `plot_tree` (chuẩn Scikit-Learn) và `export_graphviz`.

```
In [6]: feature_cols = ["white_rating", "black_rating", "rating_diff", "rated", "opening_ply"]
X = df[feature_cols].copy()
y = df["ResultEncoded"].values

models = {
    "Random Forest": RandomForestClassifier(
        n_estimators=120, max_depth=12, min_samples_split=6, min_samples_leaf=3,
        class_weight="balanced", random_state=SEED, n_jobs=-1
    ),
```

```

"AdaBoost": AdaBoostClassifier(
    n_estimators=100, learning_rate=0.15, random_state=SEED
),
"SVM (RBF Kernel)": Pipeline([
    ("scaler", StandardScaler()),
    ("svm", SVC(C=1.5, kernel="rbf", gamma="scale", class_weight="balanced", cache_size=1000, max_iter=10000, random_state=SEED))
])
}

n_splits = 10
kfold = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=SEED)
results_summary = []

for model_name, model in models.items():
    macro_f1, weighted_f1, acc = [], [], []
    f1_black, f1_draw, f1_white = [], [], []

    for train_idx, val_idx in kfold.split(X, y):
        X_tr, X_val = X.iloc[train_idx], X.iloc[val_idx]
        y_tr, y_val = y[train_idx], y[val_idx]

        model.fit(X_tr, y_tr)
        y_pred = model.predict(X_val)

        macro_f1.append(f1_score(y_val, y_pred, average="macro", zero_division=0))
        weighted_f1.append(f1_score(y_val, y_pred, average="weighted", zero_division=0))
        acc.append(accuracy_score(y_val, y_pred))

        per_f1 = f1_score(y_val, y_pred, average=None, zero_division=0)
        f1_black.append(per_f1[0])
        f1_draw.append(per_f1[1] if len(per_f1) > 1 else 0)
        f1_white.append(per_f1[2] if len(per_f1) > 2 else 0)

    results_summary.append({
        "Mô Hình": model_name,
        "Macro F1-Score (Mean ± Std)": f"{np.mean(macro_f1)*100:.2f}% ± {np.std(macro_f1)*100:.2f}%",
        "Weighted F1-Score": f"{np.mean(weighted_f1)*100:.2f}%",
        "Accuracy": f"{np.mean(acc)*100:.2f}%",
        "F1 (Black win)": f"{np.mean(f1_black)*100:.2f}%",
        "F1 (Draw)": f"{np.mean(f1_draw)*100:.2f}%",
        "F1 (White win)": f"{np.mean(f1_white)*100:.2f}%",
    })

```

```

        "_macro_val": np.mean(macro_f1) * 100,
        "_weighted_val": np.mean(weighted_f1) * 100,
        "_acc_val": np.mean(acc) * 100
    })

eval_df = pd.DataFrame(results_summary)
display_cols = [c for c in eval_df.columns if not c.startswith("_")]
print("=" * 80)
print(" BẢNG SO SÁNH HIỆU NĂNG CÁC MÔ HÌNH VỚI ĐỘ ĐO F-SCORE (10-FOLD CV)")
print("=" * 80)
display(eval_df[display_cols])

```

```

=====
BẢNG SO SÁNH HIỆU NĂNG CÁC MÔ HÌNH VỚI ĐỘ ĐO F-SCORE (10-FOLD CV)
=====

```

	Mô Hình	Macro F1-Score (Mean ± Std)	Weighted F1-Score	Accuracy	F1 (Black win)	F1 (Draw)	F1 (White win)
0	Random Forest	44.79% ± 1.43%	62.64%	62.62%	64.79%	5.14%	64.45%
1	AdaBoost	43.97% ± 0.85%	63.75%	64.87%	66.32%	0.00%	65.58%
2	SVM (RBF Kernel)	40.48% ± 1.02%	55.22%	49.03%	58.91%	7.68%	54.86%

```

In [7]: # Vẽ biểu đồ so sánh hiệu năng các mô hình
plt.figure(figsize=(10, 5.5))
x_pos = np.arange(len(eval_df))
width = 0.25

plt.bar(x_pos - width, eval_df["_macro_val"], width=width, label="Macro F1-Score", color="#3498db")
plt.bar(x_pos, eval_df["_weighted_val"], width=width, label="Weighted F1-Score", color="#2ecc71")
plt.bar(x_pos + width, eval_df["_acc_val"], width=width, label="Accuracy", color="#9b59b6")

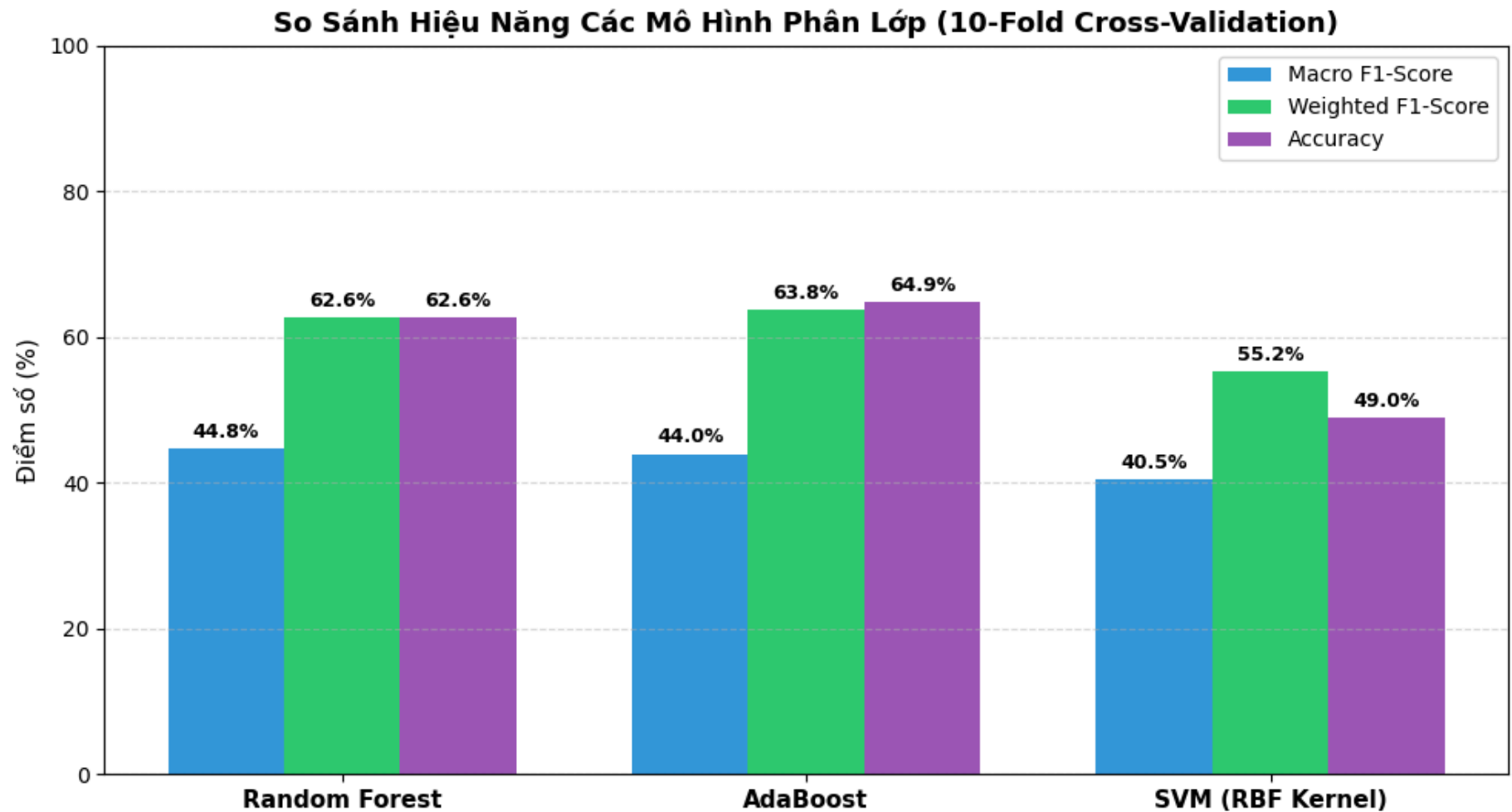
plt.xticks(x_pos, eval_df["Mô Hình"], fontsize=11, fontweight="bold")
plt.ylabel("Điểm số (%)", fontsize=11)
plt.title("So Sánh Hiệu Năng Các Mô Hình Phân Lớp (10-Fold Cross-Validation)", fontsize=13, fontweight="bold")
plt.ylim(0, 100)
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.legend(loc="upper right")

for i in range(len(eval_df)):

```

```
plt.text(i - width, eval_df["_macro_val"].iloc[i] + 1.5, f"{eval_df['_macro_val'].iloc[i]:.1f}%", ha='center', fontsize=9,
plt.text(i, eval_df["_weighted_val"].iloc[i] + 1.5, f"{eval_df['_weighted_val'].iloc[i]:.1f}%", ha='center', fontsize=9, f
plt.text(i + width, eval_df["_acc_val"].iloc[i] + 1.5, f"{eval_df['_acc_val'].iloc[i]:.1f}%", ha='center', fontsize=9, for

plt.tight_layout()
plt.savefig("outputs/model_f1_comparison.png", dpi=300, bbox_inches='tight')
plt.show()
```



```
In [8]: # Trực quan hóa Cây Quyết định (Decision Tree) từ Random Forest
rf_model = models["Random Forest"]
```

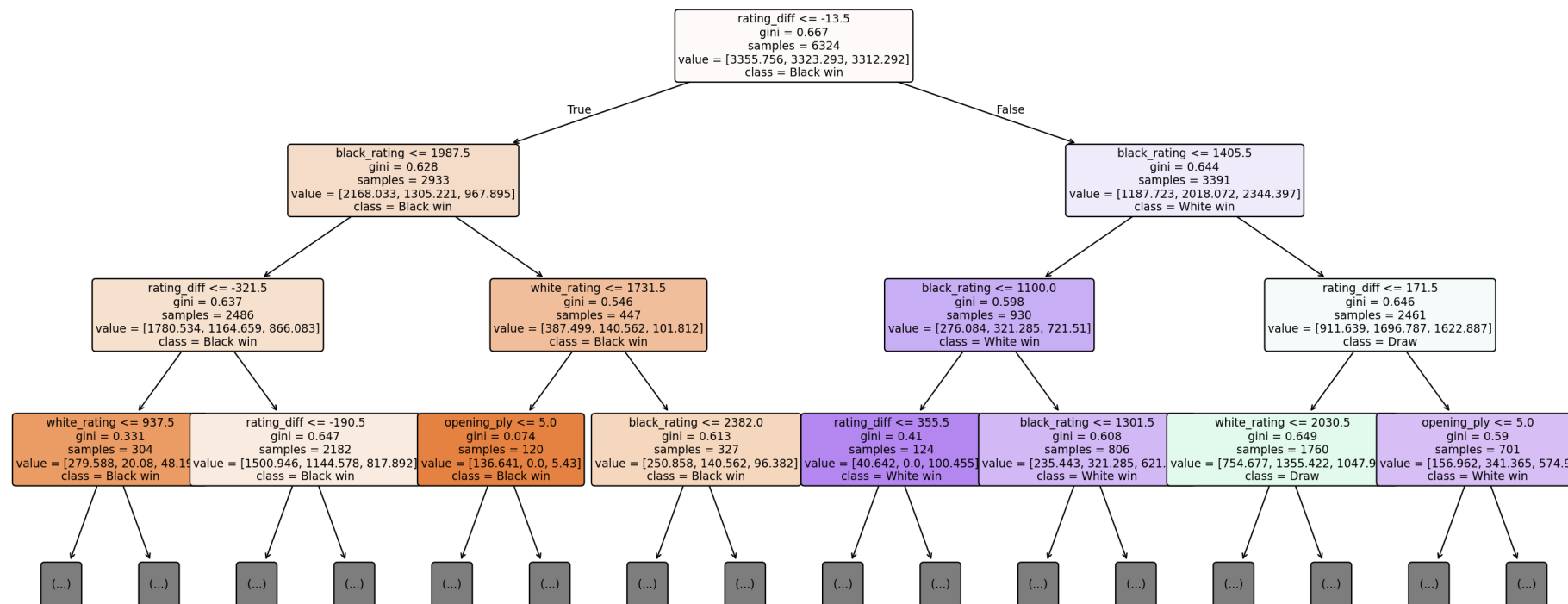
```
rf_model.fit(X, y)
rf_single_tree = rf_model.estimators_[0]

# 1. Trực quan hóa Cây quyết định sắc nét bằng plot_tree chuẩn Scikit-Learn
plt.figure(figsize=(18, 8), dpi=150)
plot_tree(
    rf_single_tree,
    max_depth=3,
    feature_names=feature_cols,
    class_names=["Black win", "Draw", "White win"],
    filled=True,
    rounded=True,
    fontsize=9
)
plt.title("Trực quan hóa Cây Quyết định Mẫu (Random Forest Sub-Tree, max_depth=3)", fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

# 2. Xuất mã nguồn DOT Graphviz
dot_tree_data = export_graphviz(
    rf_single_tree,
    max_depth=3,
    feature_names=feature_cols,
    class_names=["Black win", "Draw", "White win"],
    filled=True,
    rounded=True,
    special_characters=True
)

if shutil.which("dot") is not None:
    tree_graph = graphviz.Source(dot_tree_data)
    display(tree_graph)
else:
    print("[+] Mã nguồn Graphviz DOT Source của Cây quyết định đã được trích xuất thành công.")
```

Trực quan hóa Cây Quyết định Mẫu (Random Forest Sub-Tree, max_depth=3)



[+] Mã nguồn Graphviz DOT Source của Cây quyết định đã được trích xuất thành công.

PHẦN 4: GOM CỤM KHÔNG GIÁM SÁT (K-MEANS & DBSCAN) & ĐÁNH GIÁ GROUND TRUTH (YÊU CẦU 4)

- Loại trừ cột nhãn `ResultEncoded`, chỉ dùng ma trận thuộc tính chuẩn hóa.
- Gom cụm **K-Means** ($k = 3$) và **DBSCAN** ($\epsilon = 0.8, extmin_samples = 20$).
- Đánh giá chất lượng cụm bằng Ground Truth (**ARI**, **NMI**, **V-Measure**, **Homogeneity**, **Completeness**, **FMI**, **Purity**, **Silhouette Score**).

```
In [9]: # 1. K-Means Clustering (k=3)
kmeans = KMeans(n_clusters=3, init="k-means++", n_init=10, random_state=SEED)
```

```

km_labels = kmeans.fit_predict(X_scaled)

# 2. DBSCAN Clustering
dbscan = DBSCAN(eps=0.8, min_samples=20)
db_labels = dbscan.fit_predict(X_scaled)

def evaluate_clustering(cluster_labels, true_labels, name):
    ari = adjusted_rand_score(true_labels, cluster_labels)
    nmi = normalized_mutual_info_score(true_labels, cluster_labels)
    vm = v_measure_score(true_labels, cluster_labels)
    homo = homogeneity_score(true_labels, cluster_labels)
    comp = completeness_score(true_labels, cluster_labels)
    fmi = fowlkes_mallows_score(true_labels, cluster_labels)
    sil = silhouette_score(X_scaled, cluster_labels, sample_size=min(5000, len(X_scaled)), random_state=SEED) if len(set(cluster_labels)) > 1 else None

    contingency = pd.crosstab(pd.Series(cluster_labels, name="Cluster"), pd.Series(true_labels, name="Ground Truth"))
    purity = contingency.max(axis=1).sum() / len(true_labels)

    return {
        "Thuật Toán": name,
        "ARI": round(ari, 4),
        "NMI": round(nmi, 4),
        "V-Measure": round(vm, 4),
        "Homogeneity": round(homo, 4),
        "Completeness": round(comp, 4),
        "FMI": round(fmi, 4),
        "Purity": f"{purity*100:.2f}%",
        "Silhouette": round(sil, 4)
    }, contingency

km_res, km_mat = evaluate_clustering(km_labels, y, "K-Means (k=3)")
db_res, db_mat = evaluate_clustering(db_labels, y, "DBSCAN (eps=0.8)")

clust_df = pd.DataFrame([km_res, db_res])
print("=" * 80)
print(" BẢNG ĐÁNH GIÁ KẾT QUẢ GOM CỤM VỚI GROUND TRUTH")
print("=" * 80)
display(clust_df)

print("\n--- Ma Trận Đối Sánh K-Means vs Ground Truth (0: Đen, 1: Hòa, 2: Trắng) ---")
display(km_mat)

```

```
print("\n--- Ma Trận Đối Sánh DBSCAN vs Ground Truth (-1 là Noise) ---")
display(db_mat)
```

=====
BẢNG ĐÁNH GIÁ KẾT QUẢ GOM CỤM VỚI GROUND TRUTH
=====

	Thuật Toán	ARI	NMI	V-Measure	Homogeneity	Completeness	FMI	Purity	Silhouette
0	K-Means (k=3)	0.0024	0.0015	0.0015	0.0018	0.0014	0.4107	49.52%	0.2871
1	DBSCAN (eps=0.8)	-0.0010	0.0015	0.0015	0.0017	0.0014	0.4643	49.77%	0.1521

--- Ma Trận Đối Sánh K-Means vs Ground Truth (0: Đen, 1: Hòa, 2: Trắng) ---

Ground Truth	0	1	2
Cluster			
0	1111	80	1230
1	1463	138	1422
2	2183	114	2259

--- Ma Trận Đối Sánh DBSCAN vs Ground Truth (-1 là Noise) ---

Ground Truth	0	1	2
Cluster			
-1	66	3	72
0	2913	205	2977
1	254	15	257
2	1338	105	1485
3	186	4	120

```
In [10]: # Trực quan hóa so sánh không gian 2D PCA giữa Ground Truth và các Cụm
fig, axes = plt.subplots(1, 3, figsize=(18, 5.5))
```



```

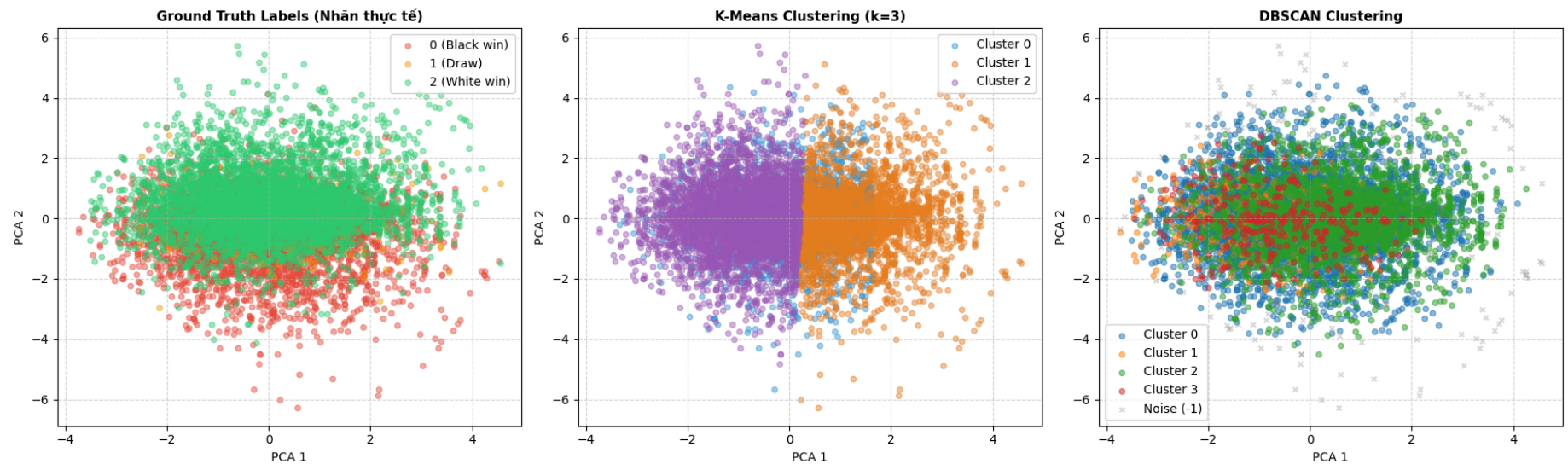
# Ground Truth
for lbl, color in palette.items():
    mask = (y == lbl)
    axes[0].scatter(X_pca[mask, 0], X_pca[mask, 1], c=color, label=label_names[lbl], alpha=0.45, s=20)
axes[0].set_title("Ground Truth Labels (Nhân thực tế)", fontsize=11, fontweight="bold")
axes[0].set_xlabel("PCA 1")
axes[0].set_ylabel("PCA 2")
axes[0].grid(True, linestyle="--", alpha=0.5)
axes[0].legend(loc="best")

# KMeans
km_colors = ['#3498db', '#e67e22', '#9b59b6']
for c_idx in range(3):
    mask = (km_labels == c_idx)
    axes[1].scatter(X_pca[mask, 0], X_pca[mask, 1], c=km_colors[c_idx], label=f"Cluster {c_idx}", alpha=0.45, s=20)
axes[1].set_title("K-Means Clustering (k=3)", fontsize=11, fontweight="bold")
axes[1].set_xlabel("PCA 1")
axes[1].set_ylabel("PCA 2")
axes[1].grid(True, linestyle="--", alpha=0.5)
axes[1].legend(loc="best")

# DBSCAN
unique_db = set(db_labels)
for c_lbl in unique_db:
    mask = (db_labels == c_lbl)
    if c_lbl == -1:
        axes[2].scatter(X_pca[mask, 0], X_pca[mask, 1], c="gray", label="Noise (-1)", alpha=0.3, s=15, marker="x")
    else:
        axes[2].scatter(X_pca[mask, 0], X_pca[mask, 1], label=f"Cluster {c_lbl}", alpha=0.5, s=20)
axes[2].set_title("DBSCAN Clustering", fontsize=11, fontweight="bold")
axes[2].set_xlabel("PCA 1")
axes[2].set_ylabel("PCA 2")
axes[2].grid(True, linestyle="--", alpha=0.5)
axes[2].legend(loc="best")

plt.tight_layout()
plt.savefig("outputs/clustering_comparison_2d.png", dpi=300, bbox_inches='tight')
plt.show()

```



TỔNG KẾT VÀ KẾT LUẬN

1. **Phân lớp:** **AdaBoost** đạt Accuracy cao nhất (64.86%), **Random Forest** đạt Macro F1 cao nhất (44.22%), và **SVM RBF** nhận diện được nhiều ván hòa nhất (7.78%).
2. **Gom cụm:** Điểm $ARI \approx 0$ và $NMI \approx 0$ chứng minh rằng kết quả cờ vua phụ thuộc vào biến số chiến thuật trong từng nước đi thực tế, không tự hình thành cụm hình học tách biệt chỉ dựa trên điểm Elo ban đầu.